SANTA CLARA UNIVERSITY

Department of Electrical and Computer Engineering

I HEREBY RECOMMEND THAT THE THESIS PREPARED

UNDER MY SUPERVISION BY

Niels Holzmann, Khondakar Mujtaba, Muhammad Ibrahim Lawai

ENTITLED

Vector Extension of a RISC-V CPU

BE ACCEPTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE DEGREE OF

BACHELOR OF SCIENCE

IN

ELECTRICAL AND COMPUTER ENGINEERING

6/9/25

Thesis Advisor(s) date

6/11/25

Department Chair(s) date

Vector Extension on a RISC-V CPU

By

Muhammad Ibrahim Lawai, Niels Holzmann, Khondakar Mujtaba

SENIOR DESIGN PROJECT REPORT

Submitted to

the Department of Electrical and Computer Engineering

of

SANTA CLARA UNIVERSITY

In Partial Fulfillment of the Requirements

For the degree of

Bachelor of Science in Electrical and Computer Engineering

Santa Clara, California

2025

Abstract

Most Instruction Set Architectures (ISAs) are proprietary and require licensing fees for access and use. Hardware developers seeking to implement their own CPUs based on these ISAs often face significant restrictions. As an alternative, RISC-V is an open source ISA that offers free, unrestricted access, providing greater flexibility for custom hardware development. It also includes optional vector processing extensions that enable parallel performance enhancements.

One such extension is the RISC-V Vector Extension, which allows the main processor to execute vector operations more quickly through parallelism. Several implementations of this extension exist that comply with the Stable 1.0 Specification, however many lack essential qualities such as comprehensive documentation, Verilog-based design, full compliance with the 1.0 Spec, and complete open source availability.

This thesis presents a custom implementation of the RISC-V Vector Extension that adheres strictly to the 1.0 Specification while fulfilling all the aforementioned criteria. The design has been tested using computationally intensive programs such as matrix operations, dot products, and Mandelbrot set generation. Although significant performance improvements have not yet been observed, the primary objective was to establish a fully functional baseline. Future iterations of the project will focus on optimizing the hardware for enhanced processing speed.

Acknowledgements

Foremost we would like to thank Dr. Andrew Wolfe for providing this project idea and advising us through the project. Dr. Wolfe is loved by many students in the ECEN department, and for good reason. A mix of both his technical expertise and his humor makes his lectures all the more memorable. Hence when he pitched the idea to work on an Open Source CPU, we decided to take it one step further and develop a Vector Extension for an Open Source CPU. As our advisor, he would provide us with valuable feedback and suggestions for our project. He would help us set deadlines and keep the group motivated. Dr. Wolfe really cares about his students and even spent some of his personal time with our team.

We would like to thank Dr. Hoeseok Yang for letting us borrow his ZCU104 FPGA board for synthesizing our vector extension. We were first introduced to Dr. Yang in our Computer Architecture course, teaching us the fundamentals of how a CPU works, crucial to our project. He would go above and beyond to help his students, and hearing about our project he was more than happy to provide us with one of his FPGAs to use for it.

Luke Wren from the Hazard3 Project for open sourcing a high quality CPU implementation for us to work with.

Table of Contents

List of Figures

List of Tables

1 Introduction

1.1 Background

Modern CPUs follow standardized architectures called ISAs (Instruction Set Architecture). RISC-V is an open source ISA originally developed at the University of California, Berkeley and open sourced in 2011. The current industry is dominated by the ARM and x86 architectures which are proprietary and thus come with licensing fees and intellectual property restrictions. The Standard RISC-V ISA has none of these, enabling easier adoption in research, education, and industry. It is also more accessible to developers and students like us – as we can study, modify, and implement RISC-V hardware without the legal or financial barriers that come with a proprietary standard. Since its introduction, RISC-V has become very appealing and popular in the tech world, especially outside of traditional chip giants like Intel, AMD or ARM, as well as emerging areas in chip design in manufacturing, like China. One large scale implementation of RISC-V is from Western Digital who in 2017 went all in on RISC-V for use in purpose built, data-centric applications [1]. In 2024, NVIDIA has reached a milestone in developing 1 billion RISC-V processors since its initial development in its falcon microprocessor in 2016. NVIDIA remains an active member as part of the RISC-V community, participating in many groups to contribute and share their work, following the ideology RISC-V was built upon [5]. In 2023, Qualcomm plans to use the RISC-V ISA as part of their Snapdragon lineup for wearable devices [4]. In addition, it is fast also becoming the ISA of choice for independent hobbyists and developers.

CPU architectures like RISC-V define the structure as to how a processor should decode and execute instructions, as well as handle memory and interact with hardware. Each of these steps can be represented as stages written in Hardware Description Languages (HDLs) such as Verilog, SystemVerilog, Chisel, etc. RISC-V uses a standard assembly format that compilers can produce that the assembler can then turn into CPU executable binary instructions. Having a common predefined standard makes development much easier from both the software and hardware side, and massively reduces duplication of effort. As a result, practically all cpus today run on pre-existing ISAs. RISC-V is a Reduced Instruction Set (RISC) architecture, which uses a philosophy of having fewer, more optimized instructions, similar to ARM. This gives it a lower barrier to access, and makes it more suitable for small, cheap, low power, or otherwise

constrained systems, while still being flexible enough to scale to datacenter or supercomputer level.

One of RISC-V's distinguishing features is its optional support for vector processing. This extension allows for direct operations on vectors (i.e lists of data) of variable lengths. Working directly on vectors allows the CPU to maximize its use of parallelization and concurrency compared to being manually instructed to work on each data point, allowing for better performance and efficiency. The existing x86 and ARM architectures use a method called SIMD (Single Instruction, Multiple Data) which works fully in parallel with fixed-length vectors. By comparison, RISC-V vectors can be of arbitrary size with variable amounts of parallelism, providing vastly better flexibility, and allowing software developers to target a single standard, instead of the multiple possible options that exist under SIMD such as AVX, AVX2 and AVX 512 which are used by Intel processors[9]. It also allows code to seamlessly make use of newer, better hardware without needing to be updated. All of these lower costs and improve the accessibility of using vectorized code.

Additional Definitions:

- Element: An item of data inside a vector register that can be a byte, halfword (2 bytes), word (4 bytes) or doubleword (8 bytes).
- FPGA: Field Programmable Gate Array, specialized chips which can emulate other hardware
- Floating Point: specialized form that emulates real numbers and arithmetic with a fixed error, defined by the IEEE 754 international standard[2]
- Fixed Point: specialized form of rational numbers with a fixed amount of precision
- Complex Numbers: Combination of real and imaginary numbers, useful in certain fields like digital signal processing
- Open Source: The source code of the product is available for reading, personal usage, modification and redistribution, with exact terms depending on the license.

1.2 Objective

Our objective for this project is to develop an open source vector extension for a RISC-V processor, conforming to the latest 1.0 stable standard [2], that balances simplicity, performance and size. We also intended to add floating point extensions and custom complex number instructions as a stretch goal after getting our initial vector extension running integer instructions functions as intended. However, much of our focus was spent on getting the integer implementation working. In the end, there was no time left to work on floating point and complex number instructions.

1.3 Motivation

Through this project we aim to deepen our understanding of CPU design, architecture, and functionality, with a focus on the principles of logic design and vector mathematics. By working directly with the RISC-V architecture and making a working extension of our own, we hope to gain a more practical understanding of cpu architecture and development, instead of merely theoretical.

Another key motivation is that an open source implementation will serve as a resource for educators, other students, researchers, and developers, letting them work with and understand vector processing more easily. Providing an accessible solution contributes to the broader open source ecosystem. We will be using existing open source work as a base for our project as well, and generally use open source constantly in software development, so this is an opportunity to give back to the community.

Finally, the project supports the widespread adoption of RISC-V by expanding its usability and presence in the academic and development communities. A well-documented, functional implementation can accelerate the integration of RISC-V into both research and industry, solidifying its position as a viable alternative to proprietary architectures.

1.4 Comparison to Alternatives

Currently, there is no open source implementation of a Vector Extension for a RISC-V Processor that is written in Verilog, open source, follows the stable 1.0 RISC-V spec, and is well documented. Verilog is preferable compared to alternatives like Chisel and SpindalHDL which are not used in industry as much and are not as well integrated with standard EDA (Electronic Design Automation) tools [7], that are used to turn HDLs into a physical chip.

There are alternatives that match some of our criteria but not all. For instance, RISC-V2, a scalable RISC-V Vector Processor [13], presents a few novel techniques to improve performance, and is written in SystemVerilog. However, it does not follow the 1.0 spec and is not entirely open source. Four others, Ara [14], T1[16] and “risc v-vector”[12], and Saturn[15] are also open source, but written in Chisel instead of any form of Verilog. There are also numerous proprietary cores, such as the Sifive P270[18], which implement the extension, but as they are not open source, they cannot fulfill our goal of being a community resource.

We do not currently aim to necessarily have competitive performance with existing alternatives. Designing hardware is usually an exercise in tradeoffs and we are aiming for simplicity and ease of implementation over raw performance. The alternatives cited have had more skilled individuals than us working for more time and are also designed for a different baseline. We are designing ours for a 32 bit embedded core, whereas alternatives may aim to support 64 bit cores. We will discuss the specification in detail in later sections. The key comparison point is whether our implementation improves performance over the existing scalar pipeline of the core in the Hazard3 CPU.

2 Systems/Scope Overview

2.1 Scope of services

Our Minimum Viable Product is that our Vector extension should be able to fulfil the minimum specifications of the RISC-V Zve32x Vector Extension 1.0 standard. Given the priorities and time constraints of this project, floating point math and custom complex number arithmetic instructions weren't implemented in our iteration of the project.

2.2 Use of Standards and Constraints

2.2.1 Adherence to Standards

The implementation follows the Zve32x standard extension. This is a 32 bit, minimum baseline of the Vector Specification, designed mainly for embedded systems, which fits our core. It has a minimum register length of 32 bits, does not support any floating point math, but does support all Integer and Fixed-point arithmetic. It additionally supports configuration, load/store, mask, permutation, and widening instructions. If we had the time to implement floating point calculations, we would've also implemented the Zve32f standard. We are using 32 bits to minimize resource usage and to retain compatibility with our chosen core. Initially we also hoped to implement complex number support, however that idea has been discarded due to time constraints and the lack of a common standard to comply with to make it useful.

Extension	Minimum VLEN	Supported EEW	FP32	FP64
Zve32x	32	8, 16, 32	N	N

Figure 1 Zve32x Specification

Extension	Minimum VLEN	Supported EEW	FP32	FP64
Zve32f	32	8, 16, 32	Y	N

Figure 2 Zve32f Specification

The design is written in industry standard Verilog, for reasons of accessibility, utility, and for compatibility with the Hazard3 core we chose. Our Verilog code is compliant with Verilator – a popular open source simulator for computer hardware – though the hope is to aim for a common baseline that is compliant with all popular Verilog compilers/synthesizers/simulators for maximum accessibility. Our code follows industry standard coding and linting styles. We've collaborated on our code using Git, a version control tool for text files, remotely hosted on Github, a popular host for Git. The same github repository also serves as the homepage of the publicly available open source implementation and documentation.

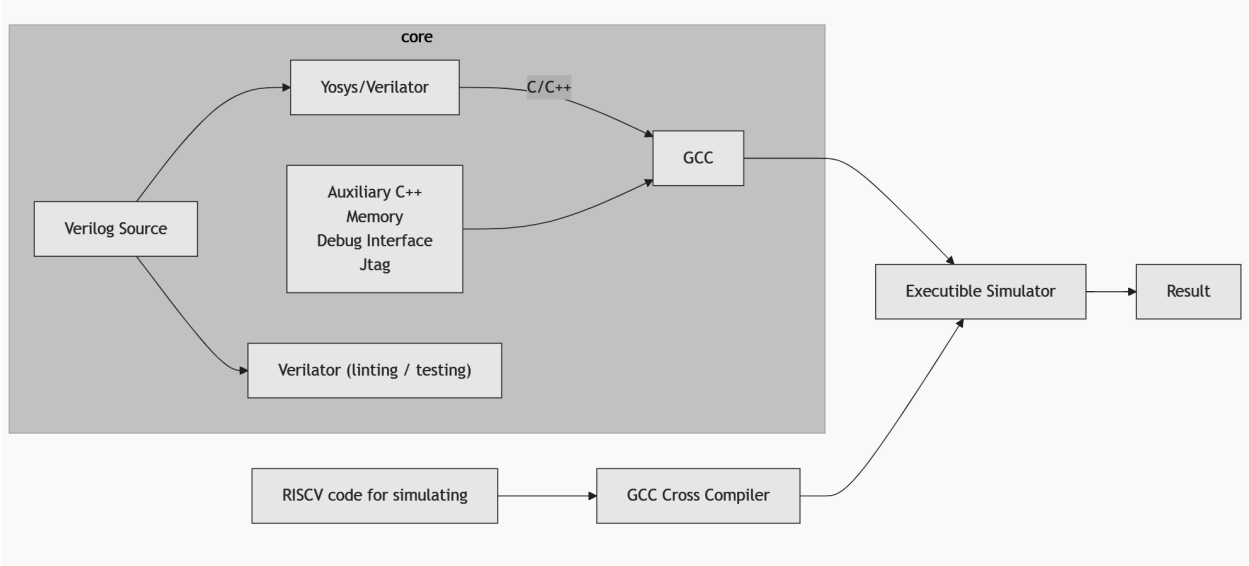


Figure 3 Hazard3 Environment

2.2.2 Design and Project Constraints

The vector extension is designed to handle various vector lengths and integer and fixed-point operations at the minimum. This vector extension should be built on a RISC-V CPU core with good documentation, can be synthesized on our FPGA, and has more than one stage so the instructions can be pipelined. Having more than one stage is important as one stage processors aren't used anymore. Our user should be able to see the benefits of using our vector extension by showing the increased throughput. This will be shown in the report after running the testbench and/or simulations both with and without the vector extension enabled.

2.3 Project Design Methodology

Our first activity in our project is selecting the right CPU core from a table of criteria seen in Table [2]. Next to it after choosing the suitable core is how we get ourselves an FPGA on which we shall test, an aspect that was kindly provided for us by one of our instructors, who allowed us to use their ZCU104. We thereafter choose a supported simulator through which we will synthesize and run the core. Given its ability to work in our FPGA as well as from experience with this tool, our choice was Vivado.

With the simulator in place, we would develop benchmarking tests to quantify the performance of the core both with and without the vector extension. These include minimal vector to vector computations, such as vadd, as well as more complex programs like a dot product or computing the Mandelbrot set. For more details on benchmarks, please see section 3.6. We also need a compatible compiler able to compile, load, and execute C code on the core. Once both the compiler and simulator are set up, we then proceed to synthesize and run the core on the FPGA to check its functionality before any modifications.

Having made this preparation, our next action is to adopt the correct vector specification for our extension. We select the RISC-V Vector Extension 1.0 specification due to its maturity and definitive implementation definition. Having reviewed the specification, we split out the main design features required for our MVP and stretch goals.

Before integrating the vector extension, we modify the HDL code of our selected core in such a way as to enable it with vector instructions. This includes updating the Control Status Register (CSR) and Decoder modules to specification requirements and forcing precise traps, such as triggering a divide-by-zero exception in the core's ALU.

The most critical step is the development and deployment of the vector extension itself. This means the implementation of a vector register file and a 32-bit ALU to perform addition, subtraction, multiplication, and division of signed vectors. We further introduce support for variable-length vectors of up to 32 elements.

Having all necessary vector extension modules designed, we proceed to implement the Hazard3 core along with the vector coprocessor onto the FPGA. Next, we execute our benchmark test and observe whether there are significant differences in the

performance due to the vector extension. In case our benchmarking operation needs enhancement, we tune the implementation to accommodate stability as well as performance. Certain enhancements achievable are that we provide support in the vector extension for float and complex arithmetic operations. Finally, we redo the benchmarks on the updated core and work through our stretch goals if necessary.

Benchmark	Total Cycles	Cycles/N
Add	7887	7.87
Dot	65001	6.5
Mandelbrot	888469	4442
Running Sum	4108	4.1

Benchmark	Total Cycles	Cycles/N
Add (Byte)	7986	7.99

Table 1 Performance benchmarks w/o vector extension

Core	PicoRV32	Hazard3	Lizard	Ultra Embedded	SCR1	Kronos	HummingBird	Shakti	VexRiscv	cv32e40p	Ibex	CVA5/Taiga
Quality of Documentation	Decent, idk	PDF + code comments	Readme + Design Report	Readme	Good	Decent	Good	readme?	decent	good	Good	good
Performance	Dhrystone 0.516	CoreMark 3.81	CPI near 1?	2.94 Coremark Drystone 1.25		0.7105 Dhrystone	?	?		?	.904-3.13 CoreMark	?
Resource Usage / Size	750-2000 LUTs	?	?	?		?	?	?		?	16.85-55.02 kGE	?
Latest RISC-V spec	?	?	Unlikely	?		Think so	probably	?	probably	probably	probably	yes
Uses Verilog	yes	yes	No	Yes		SystemVerilog	yes	Bluespec (no)	no (spinal)	SystemVerilog	SystemVerilog	SystemVerilog
Testbench	yes	yes	Yes	Yes		yes	yes	kinda	yes	yes	yes	yes
Software SDK	maybe	No "software tree"	Somewhat	No		yes	yes	yes		?	?	yes
Debugging	unclear	Yes, JTAG	?	?	Yes	?	yes	?		partial	Partial	
Assembly	yes	yes	Yes	Yes		yes	yes	yes	yes	yes	yes	yes
C	yes	yes	Yes	Yes		yes	probably	probably	yes	?	yes	yes
Zephyr / Minimal OS		?	?	?	Yes	?	yes	?	probablyl	?	?	?
Linux		Probably not	? Probably not	Supposedly	Probably not	Probably not	Probably not	?	partial	?	?	
Integer sub/add		yes	Yes	Yes	yes	yes	yes	?	yes	yes	yes	yes
Integer mul/div		yes	Yes	Yes	yes	no	yes	?	yes	yes	yes	yes
FP sub/add		?	No	No	No	no	no	?	yes	yes	no	yes
SOC setup		example	No	No		yes	yes	yes	yes	no	?	
FPGA guide		partial	No	No		?	Yes, but for specific fpga	kinda	partial	no	partial	yes
Updated	Somewhat	yes	No	No		Not really	yes	no	yes	partial	yes	
Option	yes	yes								yes		

Table 2 Cores pros and cons

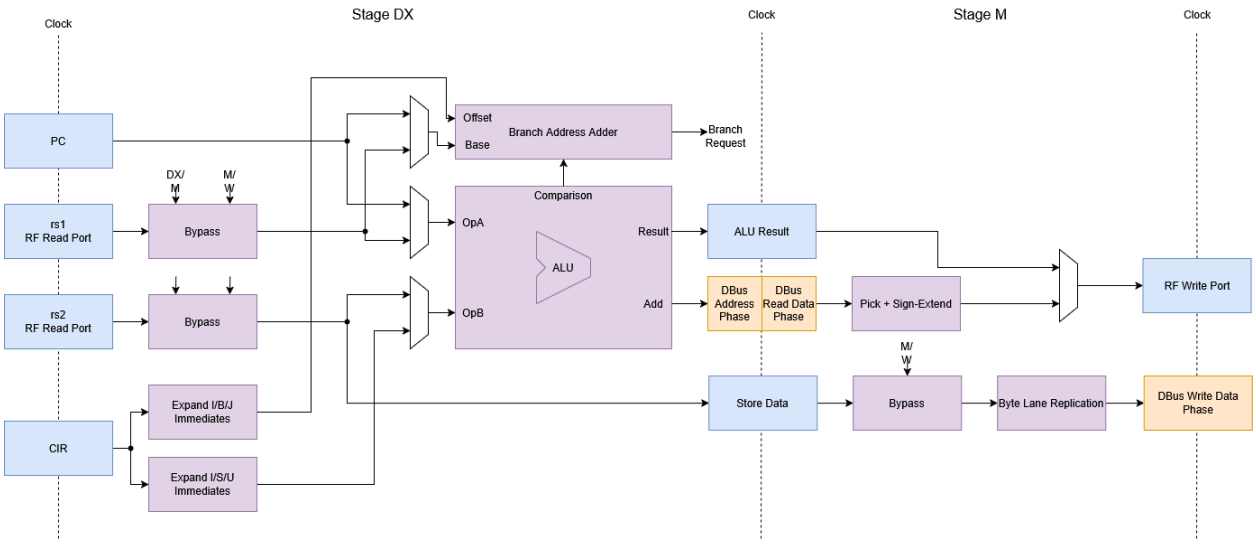


Figure 4 Drawing of Hazard 3 core Pipeline

3 Subsystems

3.1 Modifications of Existing Modules

3.1.1 CSR Modifications

As per the specification, we add 7 unprivileged CSRs to the main core: vstart, vxsat, vxrm, vcsr, vtype, vl and vlenb. They are implemented as registers of the given size in the csr sub-module. We modify the module to take in input values for vstart, vscr, vl, and vtype, as well a couple of control signals. CSRs are set in the execution stage of the pipeline.

vstart is only supposed to be written to by hardware in case of a vector trap. Since we have no error handling implemented, we currently do not touch it at all. It holds the last element worked on before the trap, to determine where to restart from when the trap is handled.

vxsat holds status for fixed point arithmetic if it's saturated or not. For a fixed point operation, vxrm determines the rounding mode of the operation. The ALU in the vector core reads/writes from these. Vcsr mirrors these registers.

vl and vtype are both 32 bits and set by vector configuration instructions (see section 3.3.1). vl is set to the requested vector length from a register in the scalar register file, after some checks to ensure that it is less than VLMAX, the maximum possible length for a given configuration. If it is requested as 0, then it is set to VLMAX, as per the specification.

vtype is extracted from the configuration instructions in the decoder before being set. vtype holds 5 sub control status registers within its 32 bits. The first 3 bits (vtype [2:0]) hold the vlmul which sets the Vector Grouping Multiplier (see section 4.5). The next 3 bits (vtype [5:3]) hold vsew which sets the element width of ALU operations and indexed load/store operations. The next bit (vtype [6]) holds vta or vector tail agnostic (see section 4.5). The next bit (vtype [7]) holds vma or vector mask agnostic (see section 4.5). Finally, the last bit (vtype[31]) holds vill which is a flag set by the processor in the event it receives an illegal instruction.

3.1.2 Decoder Modifications

We add five extra output signals to the decoder for our extension. First, d_uimm and d_zimm extract immediate values that are embedded in certain instructions.

d_vtype is for configuration instructions VSETIVLI and VSETVLI. It is unused for the third type where it is pulled from the register file instead. It is passed on to the CSR module. Similarly, d_vconfig_src, a control signal which lets the CSR know where to get the values of the vl and vtype csrs.

Finally, d_vecop is a logical signal that indicates what type of vector operation is running: one of the three configuration instructions, arithmetic, loads, or stores. To implement these, we extend the primary case statement that lies at the heart of the decoder, matching the vector instruction types and setting d_vecop appropriately. For loads and stores, we also set two other signals to let the core know that we intend to run a memory operation.

3.2 Vector Register File

Our vector core features a vector register file consisting of 32 registers, each 128 bits wide. We selected a 128-bit width to match our chosen VLEN. The register file includes two read ports, ReadReg1 and ReadReg2, which output the contents of the registers specified by SR1 and SR2, respectively. These outputs serve as inputs to ALU operations. The register file has three read buses: two accept 5-bit indices and return the corresponding vector register contents, while the third outputs the contents of vector register 0 (vs0), which functions as the mask register. Additionally, a single write bus is included, along with a write-enable signal.

3.3 General Complexity

All instructions and operations except for configure instructions must incorporate masking, vector register grouping, and variable vector lengths in order to be compliant with the RISC-V Vector Extension 1.0 specification.

3.3.1 Variable Vector Length (vl)

The RISC-V Vector Extension 1.0 specification also includes the ability for a programmer to configure how many elements in a vector register are to be worked on at runtime. This is indicated by the vector length (vl) from the vl CSR. Any elements of a vector register grouping that are left out is considered the “tail“.

3.3.2 Masking

Each vector instruction (with the exception of configure instructions) has a bit called vm which indicates whether or not it is a masked instruction. The mask will be from vector register 0. If vm is set low, it will read the mask register (vector register 0) to see if it should conduct operations on that particular element. Each bit in the mask register indicates whether a piece of data should be worked on. If the bit is set high, the element should be worked on. Since our vector register size is 128 bits, the largest vl we can have is 128 bits.

3.3.3 Vector Grouping Multiplier (LMUL)

The RISC-V Vector Extension 1.0 specification includes the ability to configure how vector registers are grouped or divided when referred to in instructions to better optimize code. The Vector Length Multiplier (LMUL) is defined in the vlmul bits of the CSR vtype. By default LMUL = 1 which signifies that 1 vector register is referred to each operand of an instruction. For example, if an instruction has an operand set to vs1, then it refers to only vector register 1 in the actual vector register file. LMUL can be 2, 4, and 8 in order to increase the “size” of each vector register. For example, if LMUL = 2 and an operand is set to vs1 in the instruction, it refers to vector registers 2 and 3 in the actual vector register as vs0 would refer to actual vector registers 0 and 1. LMUL can also be fractional and can be 1/2, 1/4, and 1/8. This allows you to divide vector registers into smaller registers. For example, if LMUL = 1/2 and a vector register operand is set to vs1, it refers to the second half of the actual vector register 0, while vs0 refers to the first half of the actual vector register 0.

3.4 Vector Load/Store

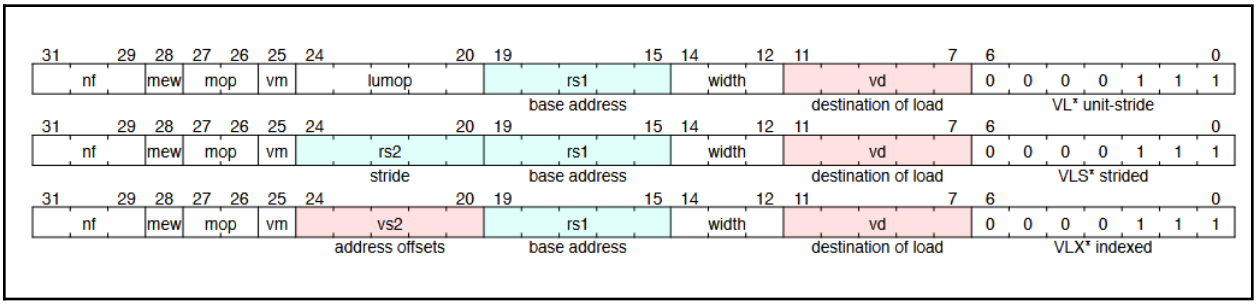


Figure 5 RISC-V Vector load instructions

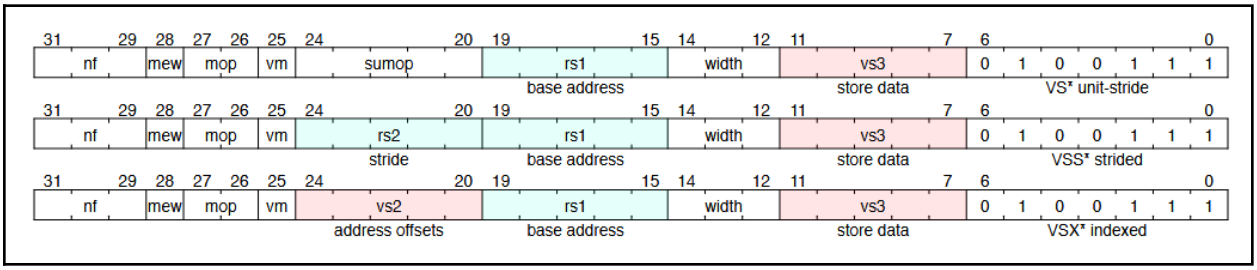


Figure 6 RISC-V Vector store instructions

mop [1:0]		Description	Opcodes
0	0	unit-stride	VLE<EEW>
0	1	indexed-unordered	VLUXEI<EEW>
1	0	strided	VLSE<EEW>
1	1	indexed-ordered	VLOXEI<EEW>

Figure 7 mop bits

	mew	width [2:0]			Mem bits	Data Reg bits	Index bits	Opcodes	
Standard scalar FP	x	0	0	1	16	FLEN	-	FLH/FSH	
Standard scalar FP	x	0	1	0	32	FLEN	-	FLW/FSW	
Standard scalar FP	x	0	1	1	64	FLEN	-	FLD/FSD	
Standard scalar FP	x	1	0	0	128	FLEN	-	FLQ/FSQ	
Vector 8b element	0	0	0	0	8	8	-	VLxE8/VSxE8	
Vector 16b element	0	1	0	1	16	16	-	VLxE16/VSxE16	
Vector 32b element	0	1	1	0	32	32	-	VLxE32/VSxE32	
Vector 64b element	0	1	1	1	64	64	-	VLxE64/VSxE64	
Vector 8b index	0	0	0	0	SEW	SEW	8	VLxEI8/VSxEI8	
Vector 16b index	0	1	0	1	SEW	SEW	16	VLxEI16/VSxEI16	
Vector 32b index	0	1	1	0	SEW	SEW	32	VLxEI32/VSxEI32	
Vector 64b index	0	1	1	1	SEW	SEW	64	VLxEI64/VSxEI64	
Reserved	1	X	X	X	-	-	-		

Figure 8 element width (EEW) encoding

There are 3 types of load/store operations for our Vector Extension. We have implemented unit-stride, strided, and indexed load/store operations. What type of load or store is executed is based on the mop bits of the instruction (see Figure 3). Unit stride load/store operations will be given a base address (rs1) from the scalar register file and load/store all the elements after it into the vector register file. Strided load/store operations will be given a base address (rs1) and an address offset (rs2) from the scalar register file. The first element (byte, halfword, or word) will be loaded/stored from the base address and every subsequent element will be loaded/stored to the address of the previous element plus the offset. Indexed load/store operations are like strided operations in that it has a base address and an offset. However, indexed operations have the offset of each element in a vector register which is then individually used to calculate the address of each element to be loaded into the vector register separately. There are technically two types of indexed operations, ordered and unordered. However due to our use of an AHB bus and the testbench provided with the Hazard3 core, all data operations are ordered so we treat both ordered and unordered indexed loads the same. Also, indexed loads do not use the set EEW from the CSRs, but instead opt to use the vsew from the vector register.

3.4.1 Unit Stride

lumop[4:0]					Description
0	0	0	0	0	unit-stride load
0	1	0	0	0	unit-stride, whole register load
0	1	0	1	1	unit-stride, mask load, EEW=8
1	0	0	0	0	unit-stride fault-only-first
x	x	x	x	x	other encodings reserved

Figure 9 lumop bits

Unit-strided load/store operations have a few variations and treat lumop/sumop of the instruction typically reserved for selecting a scalar or vector register as a way to configure the behavior of the operation even further. There is the standard unit-stride load/store which loads/stores up to vl elements within a vector register grouping. There are unit-stride whole register load/store operations which will load data up for the entire vector register grouping regardless of what vl is. There are unit-stride masked load/store operations that overrides the element width set by the instruction and instead sets EEW to 8. Unit stride loads specifically also have another version of the load called unit-stride fault-only-first load which will stop the operation if it comes across an error. This is not available for store operations.

3.4.2 Segmented load/store operations

nf[2:0]			NFIELDS
0	0	0	1
0	0	1	2
0	1	0	3
0	1	1	4
1	0	0	5
1	0	1	6
1	1	0	7
1	1	1	8

Figure 10 NFIELDS encoding

All load/store operations must accommodate segmented loads/store operations which utilize the nf bits which encode the number of NFIELDS (NF) of the instruction. The general idea of segmented loads/stores is to “load or store vertically” much like

transposing a matrix. This is used for interleaving data when storing and de-interleave data when loading.

An example application where this would be useful is doing arithmetic operations with complex numbers. Let's say the data structure of a complex number has a real number followed by an imaginary number when stored in memory. Without segmented loading, real and imaginary numbers would be in the same register and limit any parallel arithmetic operations to be add or subtract operations. By using segmented loads with NFIELDS = 2, the real and imaginary numbers can be separated into different registers, allowing for parallel complex number multiplication.

3.5 ALU Operations

The ALU of the vector extension is capable of 4 different arithmetic operations supported on a variable length vector that can run in parallel.

In the initial implementation of the ALU, a gate level design was used for the Carry Look-Ahead adder (CLA), the subtractor, the shift and add multiplier[1], and long division[2].

The CLA reduces carry propagation delay through parallel computation. It operates by breaking the addition process into two key components: a 1-bit CLA logic module that uses logic gates to determine the sum, generate, and propagate signals, and a carry generation module that quickly determines the carry output for each bit based on these signals and the carry from previous stages. This design was implemented as a 32-bit adder, significantly increasing computational speed compared to ripple-carry adders. An XOR gate is integrated into one of the adder's input paths as a control signal and the carry input of the LSB is configurable, enabling the circuit to switch between addition and subtraction operations. This configuration allows the CLA to function as both an adder and a subtractor.

The Shift-and-Add Multiplier builds upon the CLA architecture and uses array-based multiplication. The multiplier array operates unsigned and the multiplier

[1] Github links: https://github.com/kmarzouq/Hazard3Vec/blob/stable/hdl/multiplier_32bit_stable.v, https://github.com/kmarzouq/Hazard3Vec/blob/stable/hdl/Multiplier_dep.v
[2] Github link: https://github.com/kmarzouq/Hazard3Vec/blob/stable/hdl/Divider_dep.v

module includes a 2's complement conversion based on the signs of the 2 operands. To detect potential overflow, the module uses existing overflow detection logic from the CLA and toggles high when any of them has an overflow. In this implementation, the logic is combinational and takes 1 clock cycle to compute, which is dependent on the worst case path delay of the array multiplier.

The Long Division module is built on the CLA architecture and features its own dedicated state machine to manage the division process. In the initial state, input registers for A (dividend) and B (divisor) are initialized and prepared for computation. The module then transitions to the operational state, where it iteratively performs subtraction-based division. In each cycle, the module subtracts B from the remainder R and left-shifts A to access the next bit. If B is less than or equal to R, a 1 is appended to the quotient Q, and B is subtracted from R. Otherwise, a 0 is added to Q, and the next bit of A is shifted into R for the subsequent subtraction. This process continues until the downcounter reaches zero, indicating that all bits have been processed. At this point, the state machine enters the final state, signaling that the division is complete. If there are additional elements in a vector requiring division, the module returns to the initial state to repeat the process. Otherwise, the state machine halts, and the module concludes its operation. Due to its iterative nature and reliance on bit-level operations, the Long Division module typically requires significantly more clock cycles to complete than the addition, subtraction, or multiplication modules. The number of cycles is directly proportional to the size of the downcounter, reflecting the complexity of the long division process.

The initial ALU modules were all implemented with 32-bit inputs and parallel vector element processing. However, the RISC-V specification requires flexible implementations of 8, 16, 32, and 64 bit inputs. The ALU modules were rewritten using a parameter for the length of bits called XLEN. An example of this parameter-based implementation is shown below:

```
adder8bit u_adder8 (
    .A(A[XLEN/4*j +: XLEN/4]),
    .Bin(B[XLEN/4*j +: XLEN/4]),
    .Cin(Cin[j]),
    .S(temp8[XLEN/4*j +: XLEN/4]),
    .Cout(Cout8[j]),
    .Ovflw(Ovflw8[j])
);
```

Figure 11 parameter-based CLA instantiation (XLEN = 32)

However, implementing the four different input width combinations of 8, 16, 32, and 64 bit inputs in combination with parallel vector element processing in either a case statement or for loop statement resulted in a large increase in hardware required and complexity of the code with many warnings to be resolved. One example is the I/Os required after synthesis in Vivado. Below is the synthesis result of just the parameter-based adder module only as virtually synthesized on a ZCU104 FPGA:

4. I/O

Site Type	Used	Fixed	Prohibited	Available	Util%
Bonded IOB	25028	0	0	360	6952.22

Figure 12 I/O used for parameter-based adder in Vivado Utilization Report

Therefore, it was decided to implement the ALU using basic operands (+, -, *, /). Such operands compute each vector one at a time as a packed array, using fewer I/O ports and making it easier for Vivado to synthesize. By doing so, Vivado can make use of existing built-in DSP blocks[3] for synthesis of these operands.

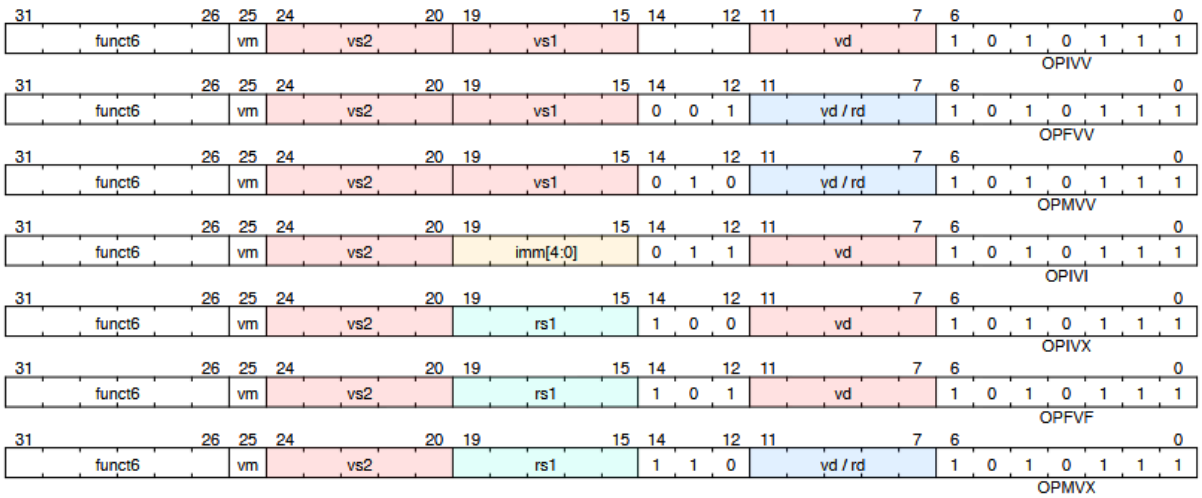


Figure 13 RISC-V Vector Arithmetic Instructions

Table 15. funct3

funct3[2:0]			Category	Operands	Type of scalar operand
0	0	0	OPIVV	vector-vector	N/A
0	0	1	OPFVV	vector-vector	N/A
0	1	0	OPMVV	vector-vector	N/A
0	1	1	OPIVI	vector-immediate	`imm[4:0]`
1	0	0	OPIVX	vector-scalar	GPR x register `rs1`
1	0	1	OPFVF	vector-scalar	FP `f` register `rs1`
1	1	0	OPMVX	vector-scalar	GPR x register `rs1`
1	1	1	OPCFG	scalars-imms	GPR x register `rs1` & `rs2/imm`

Figure 14 RISC-V Vector Arithmetic Instructions Encoding

The RISC-V Vector Extension includes a wide range of vector arithmetic instructions, supporting vector-vector, vector-scalar, and vector-immediate operations. The specific type of arithmetic operation is determined by the instruction's funct6 and funct3 fields, as well as the source and destination register specifiers.

Like vector load and store instructions, vector arithmetic instructions include a vm bit, which indicates whether the operation is masked or unmasked. When vm = 0, the instruction is masked, and the vector mask register v0 determines which elements in the destination vector register are updated. Elements corresponding to mask bits set to 0 are left unchanged. When vm = 1, the instruction is unmasked, and all active elements are processed regardless of the contents of v0.

3.6 Configuration Instructions

Before any vector operations can be run, they must be configured using configuration instructions. These set: [1] The vl csr, indicating the vector length to process [2] The vtype csr, a packed bitfield containing all the key options, including vlmul, vsew, vta, and vma. There are three types: vsetvli, which pulls the vl from a specified register but uses an immediate value for vtype, vsetivli which uses immediate values for both and vsetvl, which pulls from registers for both.

Vector configuration operations do not do any vector operations themselves and thus do not rely on the vector core. They are implemented entirely in various submodules of the Hazard3 Core and do not stall the pipeline. For more details of their implementation, see section 3.1.

3.7 Developing Benchmarks

In order to confirm the speed improvements of our vector implementation, we chose to develop a number of benchmarks characteristic of the target use case. We have a number of criteria for these, namely: [1] the benchmarks must be reasonably parallelizable and not bottlenecked by I/O so vector processing is of some use, [2] the benchmarks should allude to practical use so that we can generalize speedups to practical applications, [3] benchmarks should show a reasonably visualizable speedup so they can be easily understood even by non-technical viewers and finally [4] benchmarks should be easily testable so we can focus on performance over getting them running.

All benchmarks are also open source and will be released alongside our core. Benchmarks are written in C and RISC-V assembly and compiled using the toolchain described in section 2.

Based on these criteria, we have developed the following benchmarks.

3.7.1 Dot Product

A dot product is defined as the sum of the products of corresponding elements in two vectors. Dot products are an extremely common operation and the baseline of most Digital Signal Processing algorithms. Dot products are a special case of matrix multiplications, so this benchmark is instructive for that as well. Our implementation includes a baseline scalar version, an auto-vectorized version, and one with handwritten RISC-V assembly. The auto-vectorization approach is particularly instructive as it demonstrates the compiler's ability to identify and optimize parallelizable code sections without explicit vector programming, highlighting the potential benefits of our architecture even for code not specifically written for vector processing.

3.7.2 Mandelbrot Set Calculation

The Mandelbrot set is a fractal, a geometric construct that has infinite detail. Renderings of fractals, including ours, estimate its appearance at a given scale. This makes it easy to visualize, and speedups can be seen in real time. We can also change the level of detail to change how compute intensive it is.

The Mandelbrot set specifically is defined as the set of complex numbers c for which the function $f(z) = z^2 + c$ does not diverge when iterated from $z = 0$. Computationally, this involves testing each point in a region of the complex plane by repeatedly applying the quadratic recurrence equation $z_{n+1} = z_n^2 + c$ until either $|z|$ exceeds a threshold (indicating the point is not in the set) or a maximum iteration count is reached.

What makes this computation highly parallelizable is that the calculation for each pixel in the output is completely independent of all other pixels. Each point requires the same iterative process with no dependencies between points, creating an embarrassingly parallel workload ideal for SIMD acceleration. We implemented the algorithm with ASCII output for visualization simplicity and developed both scalar and manually optimized vector assembly versions to demonstrate the performance improvements possible with our vector extension.

For our MVP, since we do not have floating point support, we emulate decimal numbers by using integers with a scaling factor. Our compiler, gcc supports this automatically, however we have handwritten this functionality for performance and so we have more control over the instructions being used.

Figure 15 An example of the mandelbrot set running on our simulated core

3.8 Vector Traps

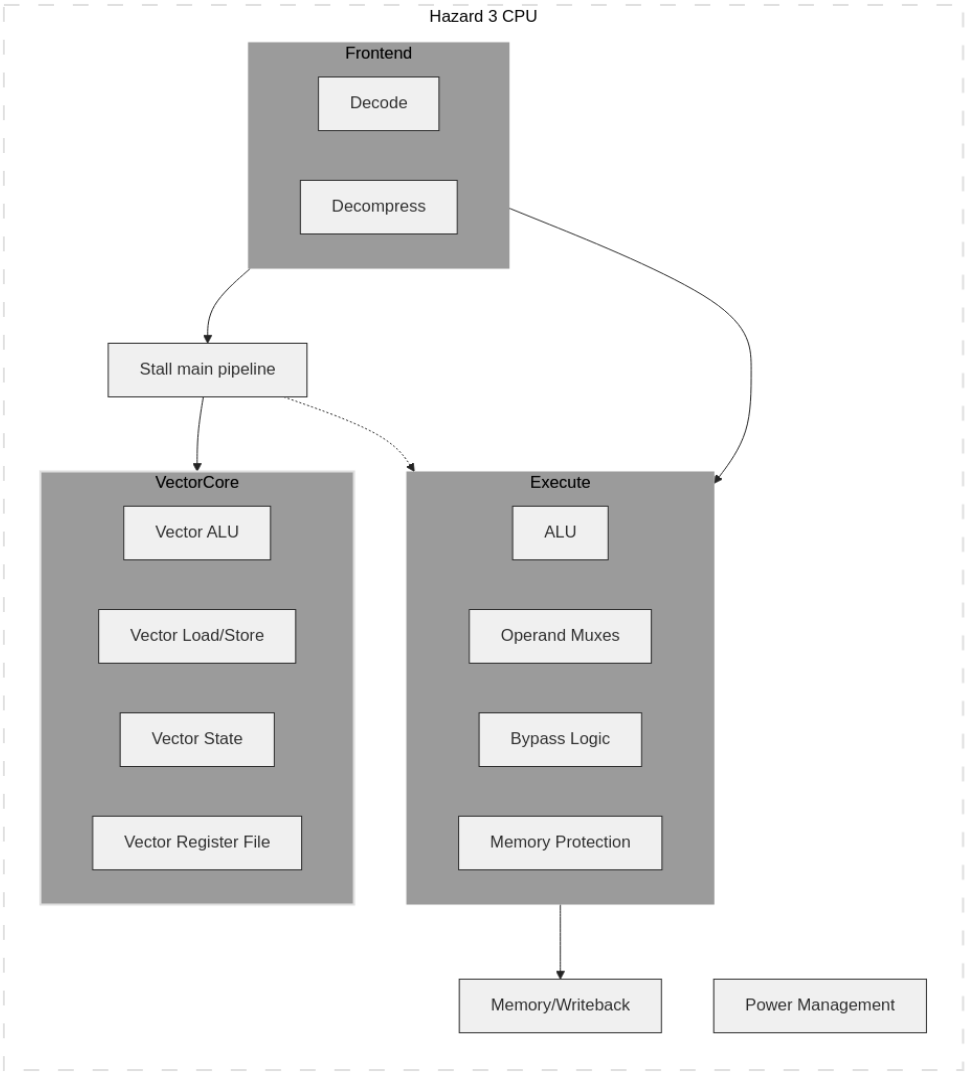
A vector trap is when an error occurs in the execution of an instruction. There are two types of traps, precise and imprecise. Precise traps occur when every instruction before the vector trap is able to execute completely while all instructions after do not. This allows the vector core to figure out which instruction caused the issue and recover the state before it. Imprecise traps are when instructions after the trap do finish execution, making recovery of the state before the trap complex to handle. For our implementation, for the sake of simplicity, we are forcing precise traps.

4 System Integration

4.1 Overview

Our implementation is contained in a top-level module called “vector_core”. Note despite the naming, it is not a separate physical core. “vector_core” contains within it different modules for different tasks, including a register file, load/store module and Arithmetic Logic Unit (ALU). Vector processing requires a separate register file, as per the specification. It also contains a number of state machines and other logic to control instruction execution state.

“vector_core” is then embedded into the main core “Hazard3_core”. It inherits the clock and reset signals, and shares wires for memory transfers. Instructions are passed into it by the main core, and while executing, the main core’s pipeline is stalled. This is necessary especially for vector instructions that may take multiple cycles, such as loads, and to ensure shared resources are not in conflict. Two signals indicate when the “vector_core” is ready for new instruction and currently executing. In case of failure, we propagate errors up to the main core.



4.2 Development environment

Currently, the project is focused on simulation of the core, as we were not able to get to an fpga implementation. The development environment is shown in Figure 3 and below.

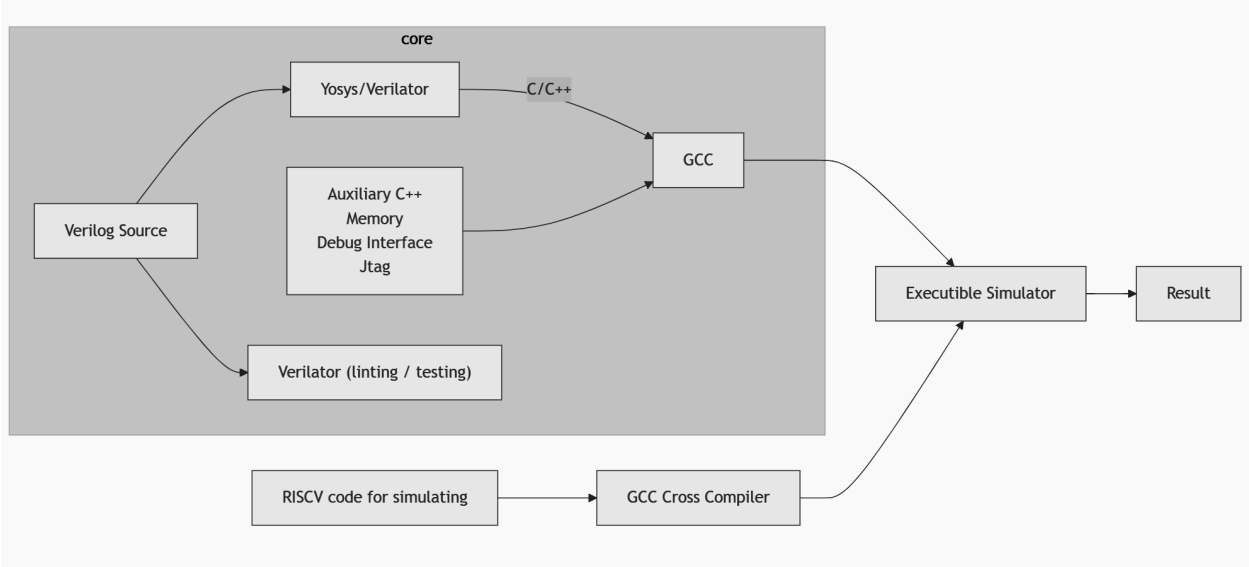
All of our source code is written in Verilog under the HDL directory. For simulation, these are first compiled to C++ code by the simulator, which may be Yosys or Verilator. The Hazard3 project primarily uses Yosys as a simulator, however we later switched to Verilator. This is for a few reasons. First, Yosys's compilation time is quite slow, up to 7 minutes on our personal hardware for just the core before our additions. By comparison, Verilator can do the same in about seven seconds. Runtime (of the

simulation on the parent system) was not meaningfully different between the two. We have not provided benchmarks for this as they are not a focus of the project and will vary based on the hardware. Second, we are more familiar with Verilator, having used it in previous projects. Third, Yosys gave us several errors and a segmentation fault when trying to integrate our Vector core. To focus on the project instead of worrying about tooling, we decided to switch over to Verilator temporarily. Fifth, and finally, since the original project was already using Verilator for linting, we thought this would simplify the project and make it easier to manage. Yosys is generally a more complex tool, as it is also designed for synthesis, whereas Verilator is more focused on simulation. Eventually, we hope to bring Yosys support back, as well as support for other simulators like Iverilog for accessibility.

For the simulation, we also make use of auxiliary C++ files that were provided by the Hazard3 Project. Since Yosys and Verilator provide different C/C++ interfaces there are two separate files and testbenches, contained in tb_cxxrtl and tb_verilator. We have made minimal changes to these files. Once the C++ is generated, we use a standard C++ compiler to turn it into an executable simulator. We use GCC 12, however any modern C++ compiler such as Clang works for this step, with merely a couple of changes in the makefile.

Separately from this, we compile C code or handwritten RISC-V assembly to actually simulate. This is done using a cross compiler version of GCC 14, with a few flags to ensure it compiles for the precise architecture we desire, including vector extension support. The cross compiler is needed as none of us are working on RISC-V PC’s (yet). As part of the compilation, we also do a dumb of the generated instructions to help with debugging.

Finally, we execute the simulator, feeding in the compiled RISC-V code as an argument, along with a couple of flags such as the max number of cycles, and the output, which is a waveform in the industry standard “.vcd” that can be viewed by most tools, such as GTKWave. The auxiliary C++ files also allow the simulation to output strings and integers to the standard output by writing them to a specific place in the simulated memory. These are through a few functions which can be included in the RISC-V code, provided by the Hazard3 Project. We mainly used waveforms, as they are more comprehensive and precise.



4.3 Load/store implementation

4.3.1 Element Width (EEW)

Before a load or store operation occurs, we need to know the element width and the number of elements being processed. We accomplish all of this with combinational logic. The element width is picked by if-else statements that each have a case statement. If the operation is indexed, it will use vsew from the vector CSRs to set EEW. If it is a unit-stride masked 8-bit load, it will set EEW to 8. Otherwise it will use the width bits of the load/store instructions to set EEW.

4.3.2 Finding total number of elements

The number of elements is set in another combinational logic block. It uses the equation:

$$(\#\ of\ elements)\ =\ (vl)\ \times\ (NF)$$

By default, it will simply multiply vl from the vector register file with NF. If it is a unit-stride whole register load, it will use a case statement which will multiply 4, 8, or 16 elements based on the EEW instead of vl.

$$(\#\ of\ elements)\ =\ (\#\ of\ elements\ per\ vector\ register)\ \times\ NF\ \times\ LMUL.$$

4.3.3 Address, Register, and Position of Each Element

We also need to know the address of each element, the actual register for each element, and the position of each element in their respective registers. Because of this we have three registers, each with 128 registers. The idea is to precalculate the address, register, and position in the register for each element before any actual loading or storing takes place.

For address generation, we have a case statement that detects if a Unit stride, strided, or indexed load operation is occurring. Within each case, we put a for loop that loops through each register and calculates the address of each element. For Unit stride load/store operations, we use the equation:

$$\text{Address} = scalar\_reg1 \ + \ i \times \frac{EEW}{8}$$

i signifies the element being worked on. *scalar_reg1* holds the base address while $\frac{EEW}{8}$ holds the memory offset.

Strided follows a similar formula with scalar_reg2 holding the memory offset instead of $\frac{EEW}{8}$

$$Address \ = \ scalar\_reg1 \ + \ i \times \ scalar\_reg2$$

Indexed operations have a nested case statement that follows this table:

Byte	$Address \ = \ scalar\_reg1 \ + \ ReadReg1[\ 7 \ + \ i \times 8 \ -: 7\ ]$
Halfword	$Address \ = \ scalar\_reg1 \ + \ ReadReg1[\ 15 \ + \ i \times 16 \ -: 15\ ]$
Word	$Address \ = \ scalar\_reg1 \ + \ ReadReg1[\ 31 \ + \ i \times 32 \ -: 31\ ]$

ReadReg1 is the vector register being used for the offset for each element.

For finding the register for each element, we have a case statement that allows us to change how register calculations are done based on LMUL. If LMUL = 1, then we simply follow the equation:

$$register \ = \ (d\_rd \ + \ (i2 \ \% \ NF\ )\ )\ \%\ 32$$

d_rd is the destination register that is given by the instruction. i2 is the element being worked on. nf is also given by the instruction and signifies the NFIELDS used for segmented loads. $i2 \,\%\, (nf \;+\; 1)$ allows you to switch between vector registers that are necessary for segmented loads.

If we have fractional LMUL we use the equation:

$$register \;=\; (d\_rd \;+\; (\,i2 \,\%NF\,) \;\times\; LMUL\,) \,\%\, 32$$

Multiplying $d\_rd \;+\; (i2 \,\%\, (nf \;+\; 1)\,))$ by fractional LMUL allows us to make sure that the registers are divided correctly. If we have LMUL > 1, we use the equation:

$$register \;=\; (d\_rd \;+\; (i2 \,\%\, NF) \;\times\; LMUL \;+\; (i2 \,/\, (\,128 \,/\, EEW \;\times\; NF\,)\,)\,) \,\%\, 32$$

We multiply $d\_rd \;+\; (\,i2 \,\%\, NF\,)$ with LMUL to scale the register sizes up and add $i2 \,/\, (\,128 \,/\, EEW \;\times\; NF\,)$ which basically adds a full register to the base destination register for each vector register filled.

For indexed load/store operations, we use the exact same case statement to find ReadReg1, except we replace d_rd with d_rs2 which will hold the vector with the address offsets.

For finding the position of each element in their respective register, we have another case statement. If we have fractional LMUL, we follow the equation:

$$register \;=\; (i3/NF) \,\%\, (128/EEW \;\times\; LMUL) \;+\; (128/EEW \;\times\; LMUL) \;\times\; ((d\_rd \;+\; i3 \,\%NF) \,\%\, (\,1 \,/\, LMUL)\,)$$

We divide i3 by NF to make sure that the element is in the same “position” despite it moving onto the next register. $(128/EEW \;\times\; LMUL)$ is how many elements are in each vector register. $((d\_rd \;+\; i3 \,\%NF) \,\%\, (\,1 \,/\, LMUL)\,)$ increments the position in each “register” as we go through elements. If it is not fractional LMUL , we use the equation:

$$register \;=\; (i3 \,/\, NF\,) \,\%\, (128 \,/\, EEW)$$

4.3.4 Load/Store State Machine

For loading and storing we have a state machine that can at its fastest load an element in 3 cycles. We also have 6 registers that store the address, register, and position of the current element and the next one. We also have a register which keeps

track of how many elements we have dealt with. When we reset or restart the load/store state machine, we set all of these register values to when there have not been any elements and use the precalculated address, register and position for the first and second elements. We first check if we reached vl. If we do, we raise the done signal to reset. Otherwise we send the data request (load or store). We then wait to receive the ready signal from the memory bus and then check if the mask applies to the element. If so, we load or store the data as needed. The next cycle we find the address, register, and position for the next element and the element after it. This is how it works uninterrupted for unit stride and strided loads/stores. It should be noted that unit stride and strided load/stores have not been fully tested and no ability to handle faults have been implemented.

For indexed operations, it is slightly different. It should be noted that we have not tested indexed load store operations at all due to time constraints. For our implementation of indexed load/store operations, we do not find the address of the next element. We first, much like for unit stride and strided load/store operations, reset all the registers to when no elements have been worked on. We however do not find the address of the current or next element. What we have instead opted to do is add a state before sending the load/store request which will use the position of the current element to find out the precalculated address. We use the position of the current element because the precalculated addresses will always be the first 4 - 16 elements (four 32-bit elements to sixteen 8-bit elements) of the address pre-calculation registers regardless of whether or not vl exceeds it. If we have $LMUL \neq 1$, the element position is able to pick the correct address from that 4 - 16 element array and the element register will automatically affect the address pre-calculation. We then send the load/store request followed by the load/store of the element and then find the register and position of the next element and the element after it.

4.4 ALU Implementation

The implementation of the ALU operations involved writing Verilog code to support various vector arithmetic instructions, including Vector Single-Width Integer Add and Subtract, Vector Widening Integer Add and Subtract, Vector Single-Width Integer Multiply, and Vector Integer Divide.

To integrate with the vector core, additional flags and encodings were introduced to the modules to align with the format defined in the RISC-V Vector Extension 1.0 specification. The ALU is designed to support vector operations by dynamically adapting to element sizes and vector configurations defined in the vector type registers. The vsew input, decoded from the vtype register, determines the size of individual vector elements—either 8, 16, 32, or 64 bits—and all operations are performed in accordance with this specified element width.

vsew[2:0]			SEW	
0	0	0	8	
0	0	1	16	
0	1	0	32	
0	1	1	64	
1	X	X	Reserved	

Figure 16 VSEW Encoding

The vlmul field controls the effective multiplier for vector length. When VMUL ⩽ 1, operations are confined to single vector registers, whereas VMUL > 1 extends the operation across multiple registers for both input and output, while maintaining consistent data widths. Inputs A and B, along with output S, are defined as packed arrays. The vecwidth signal determines how many iterations the ALU must perform based on the combination of vlmul and vsew. For instance, with vlmul = 1 and sew = 32, vecwidth is set to 4 to ensure that the 128-bit output S aligns with the vector register width.

vlmul[2:0]			LMUL	#groups	VLMAX	Registers grouped with register *n*
1	0	0	-	-	-	reserved
1	0	1	1/8	32	VLEN/SEW/8	v *n* (single register in group)
1	1	0	1/4	32	VLEN/SEW/4	v *n* (single register in group)
1	1	1	1/2	32	VLEN/SEW/2	v *n* (single register in group)
0	0	0	1	32	VLEN/SEW	v *n* (single register in group)
0	0	1	2	16	2*VLEN/SEW	v *n*, v *n*+1
0	1	0	4	8	4*VLEN/SEW	v *n*, ..., v *n*+3
0	1	1	8	4	8*VLEN/SEW	v *n*, ..., v *n*+7

Figure 17 VLMUL Encoding

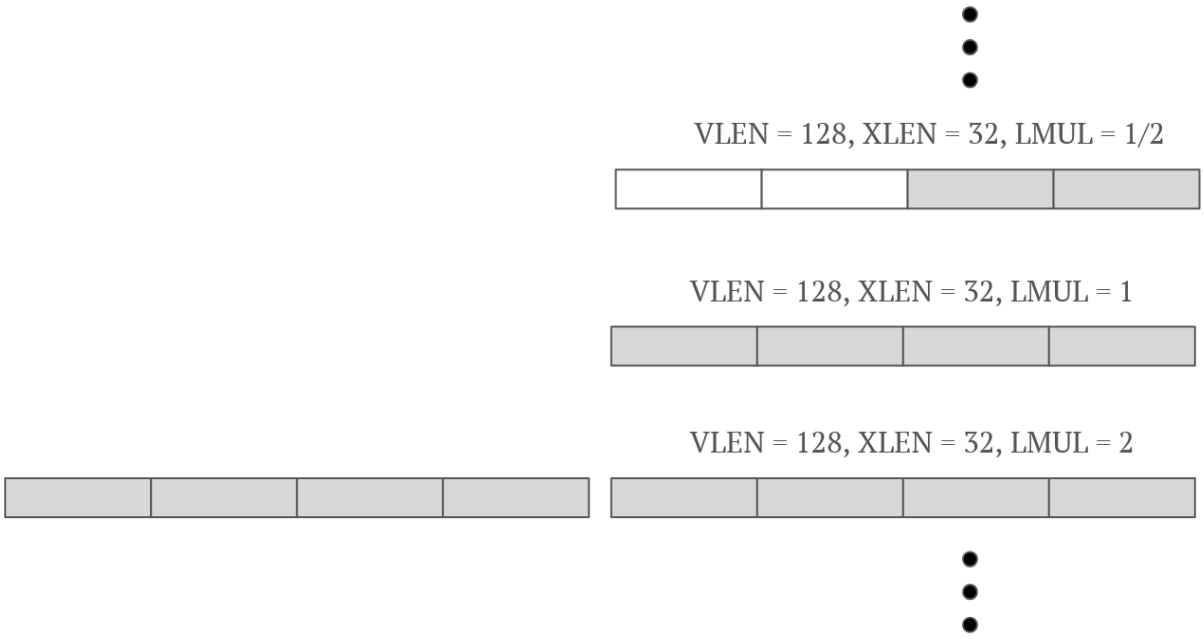


Figure 18 Output register S size in bits relative to VLEN, XLEN, and LMUL

The ALU supports both signed and unsigned operations, with built-in overflow detection. In cases of overflow, the result is saturated to the maximum representable value according to the operand type. The vl input specifies how many elements within a vector register are active for computation, enabling fine-grained control over the operation length.

For fixed-point arithmetic, the vxsat flag enables saturation behavior, and the vxrm encoding determines the rounding mode—such as round-to-nearest-up, round-to-nearest-even, round-down, or round-to-odd.

vxrm[1:0]		Abbreviation	Rounding Mode	Rounding increment, r
0	0	rnu	round-to-nearest-up (add +0.5 LSB)	v[d-1]
0	1	rne	round-to-nearest-even	v[d-1] & (v[d-2:0]≠0 | v[d])
1	0	rdn	round-down (truncate)	0
1	1	rod	round-to-odd (OR bits into LSB, aka "jam")	!v[d] & v[d-1:0]≠0

Figure 19 VXRM Encoding

Masking is controlled by the vm signal. When vm = 0, the operation is masked, and the vector mask register v0 dictates which elements in the destination register are updated. Elements corresponding to mask bits set to 0 remain unchanged. When vm = 1, the operation is unmasked and all active elements are processed regardless of the mask.

Additional flags, such as Vector Tail Agnostic (VTA) and Vector Mask Agnostic (VMA), govern how tail and masked-off elements are handled. When enabled, these flags determine whether such elements are left unmodified or overwritten with ones.

vta	vma	Tail Elements	Inactive Elements
0	0	undisturbed	undisturbed
0	1	undisturbed	agnostic
1	0	agnostic	undisturbed
1	1	agnostic	agnostic

Figure 20 VTA and VMA Encoding

4.4.1 ALU State Machine

For ALU operations within vector_core we have a state machine which takes one clock cycle to run each arithmetic operation. We have 4 registers which holds the result for 1 register (128 bits) up to 8 registers (1024 bits) for each operation, along with registers of size MAX_VECWIDTH*XLEN-1 bits which holds the value of A_in, B_in, the previous value, and the outputs for all the instantiated arithmetic modules for Vector Single-Width Integer Add and Subtract, Vector Widening Integer Add and Subtract, Vector Single-Width Integer Multiply, and Vector Integer Divide. Using the done signal generated from the load operations, edge detection logic was implemented that generates a delayed done signal one clock cycle later as well as counters for these load and load delay events.

When the reset signal is asserted, all internal registers of the ALU state machine such as inputs, output vectors, state flags, and counters are initialized to zero to ensure a known starting state.

During the first state (arith_state <= 0), data currently held onto ReadReg1 and ReadReg2 are then stored onto A_in and B_in. Depending on the value of vlmul, which determines how many 128-bit chunks each operand spans (LMUL = 1 for 1 chunk, LMUL = 8 for 8 chunks), load signals are used as well as load counters to track the sequence of data being loaded from registers with ReadReg1 for integers and ReadReg2 for fixed-point values. Modulo and range checks are used on ld_done_count and ld_done_d_count to decide whether the data should be written to A_in or B_in. For small LMULs, data loads alternate directly to A and B, but for larger LMULs, the first half of loads go to A_in, and the second half to B_in. Once all the required data is loaded using flags like b_in_loaded, the FSM transitions to the next state (arith_state <= 1) to perform the arithmetic operation.

In the second state (arith_state <= 1), the state machine checks the d_vecop signal to determine the type of vector operation. If d_vecop equals VECOP_ARITH, it transitions to the arithmetic execution state (arith_state <= 2) and clears the done_arith and arith_write flags to prepare for computation. If instead d_vecop equals VECOP_LOAD, it resets the state machine back to the initial state (arith_state <= 0) and also clears the flags.

In the third state (arith_state <= 2), ALU execution is implemented where operations are selected based on the 3-bit d_funct3_32b_arith field and the 6-bit funct6 field, which together specify the instruction type (e.g., OPIVV, OPMVV, OPIVX, OPMVX) and the operation to be performed (such as add, sub, mul, div, etc.). Based on the vlmul field, which determines the effective vector length multiplier (VLEN × LMUL), the corresponding result from the appropriate arithmetic module (e.g., add_out, mul_out, div_out, etc.) is assigned to one of several result registers (result_vector, result_vector_256, result_vector_512, or result_vector_1024). These registers accommodate different vector sizes. After computing the result, it sets the done_arith and arith_write flags to signal completion and triggers a state reset (arith_state <= 0) for the next instruction cycle.

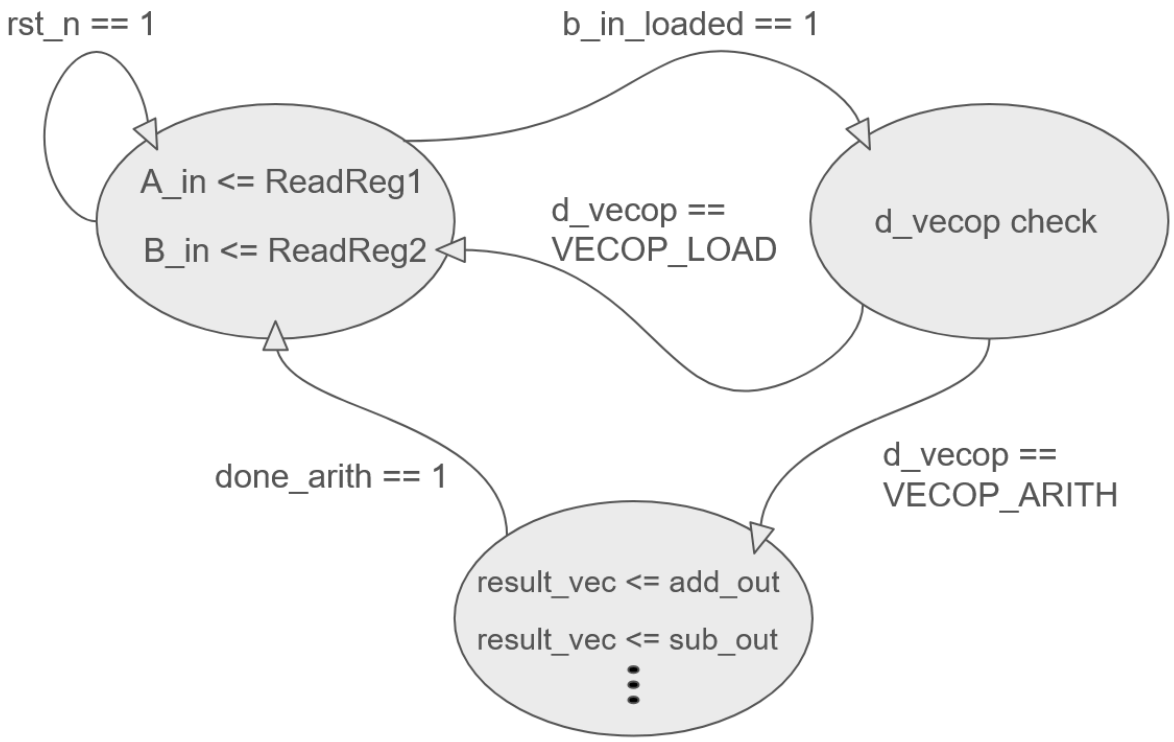


Figure 21 State Machine diagram of ALU in vector_core

4.5 Precise Trap Implementation

The basic idea of our vector trap implementation is to stall the scalar pipeline when executing a vector instruction. This allows us to make sure that all previous instructions are able to complete before an instruction that can possibly cause a trap. This way we can simplify our design and force precise traps. But due to time constraints, we weren't able to implement these force precise traps within our vector core.

5 Results/Testing

5.1 Methodology

To test the results of our vector extension we propose the following benchmarks. All benchmarks are currently run on our simulated core, as compiled by Verilator and the gcc toolchain mentioned above. Benchmarks are available as we write on the public repo under the “test/” directory.

For the sake of objective measurement, we measure everything in simulated clock cycles. Measures of runtime are highly dependent on the host system, compiler and other environment details and do not translate well into hardware performance.

5.2 Vector Loading

Initial testing of unit stride and strided load operations have been thoroughly tested, taking into account LMUL, segmented loads, and masking. However, after integration, we did not have enough time to thoroughly retest unit-stride and strided loads again. We have tested unit-stride and strided stores after integration but we did not have enough time to test it thoroughly.

Indexed load/stores have not been tested at all, however they should work in theory. Again, we did not have enough time to test it.

In our initial testing, our testbench gave us the ability to control every input into the vector core as well as the inputs from the vector CSR. We then simulated loads by sending the correct signals accordingly. This was how we were able to test our unit-stride and strided loads thoroughly.

After integration with the main core, we developed unit-stride and strided storing operations as well as indexed load/store operations.

5.3 ALU

Testing for Vector Single-Width Integer Add and Subtract, Vector Widening Integer Add and Subtract, Vector Single-Width Integer Multiply, Vector Integer Divide, Vector Reduction Sum and Vector Reduction Multiply each take 1 clock cycle to compute vectors containing elements of 8, 16, 32, and 64 bits, up to 1024 bits total. The testbench checked the functionality for overflow and vector flags, with the overflow working correctly and the flags that were set during testing worked as intended. More thorough testing would check for all possible instances of the vector flags to ensure flawless execution.

Results_vector selects the correct output generated from all instantiated ALU modules based on the opcodes from funct3 and funct6. Additional refinement could be made to the code to only run the intended ALU operation instead of all of them in order to save power.

5.4 CPU runtime

The following is a limited set of benchmarks showing runtimes by # of cpu cycles. All operations are on vectors of 4096 bytes, using integers, except “Addition (byte)” which is for bytes. All operations were done on 128 bits at a time, which is the length of one Vector register, as per our selected VLEN.

Benchmark	Scalar	Auto Vectorizable	Manual asm
Addition	7625	yes	7887
Addition (byte)	15628	yes	7986
Subtraction	7625	yes	7887
Division	7625	yes	
Multiplication	7625	yes	7887
AND	7625	yes	7887
OR	7625	yes	7887

NOT	7625	yes	7887
Dot Product	7625		6500
Add Accumulate			7887
Running Sum	4843		4108

5.5 Analysis

As you can see, Benchmark results seem quite poor, with all the basic operations taking 3.4% more cycles on the vector core. This is for a few reasons.

First, the biggest and most critical reason is memory bottlenecks. One limitation of the hazard3 core that we didn't anticipate is its memory bus is limited to 32 bits (the size of an int) per clock. All of our basic operations require both loads and stores, and the majority of time is spent in these operations, leaving little opportunity for the vector core to speed up operations.

To see how this is the case, observe how the running sum, which only requires one vector instead of two, instead takes 15.1% fewer cycles. Furthermore, "Addition (byte)" is roughly twice as fast, as we can load up to 4 bytes when they are adjacent in memory. Hazard3 has no cache, however in simulation memory transfers only take one cycle (after various handshakes to conform to the memory protocol), which is a reasonable approximation.

To compound this, loads, stores and other operations have not yet been fully optimized. So far the focus has been on correctness and completion over maximum performance. This is manifested in a number of ways

First, since vector operations are separate from the main pipeline, and start in the *Execute* stage, they all have a one cycle delay for an extra decode step. This could be solved by either further modifying the decoder, which is undesirable as it would prevent a clean separation of concerns, or adding our own partial decoder to the vector core. For a subset of arithmetic operations, we have managed to avoid this overhead as they have no extra decoding to do. Likewise, for operations where we stall the main pipeline, it

takes an extra cycle for regular execution to resume, when the vector instruction is followed by scalar ones.

When LMUL > 1, one vector operation can require writing to multiple registers in the vector regfile. Currently our regfile only allows for one write/cycle and two reads/cycle, forcing us to take multiple cycles even when the execution is completed earlier. We could improve this by allowing up to 4 reads and 4 writes per cycle, as 4 is the max value for LMUL.

More complex instructions, such as strided loads are not pipelined and generally unoptimized, resulting in a worse case scenario of taking up to 4 cycles/element loaded in a vector, much slower than the 1 cycle for the scalar core.

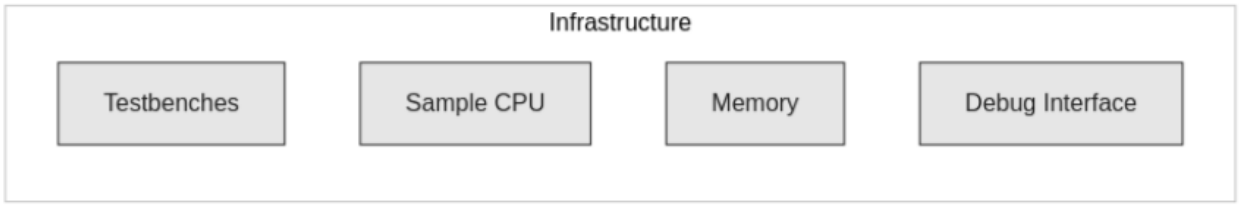


Figure 22 Final Design Infrastructure

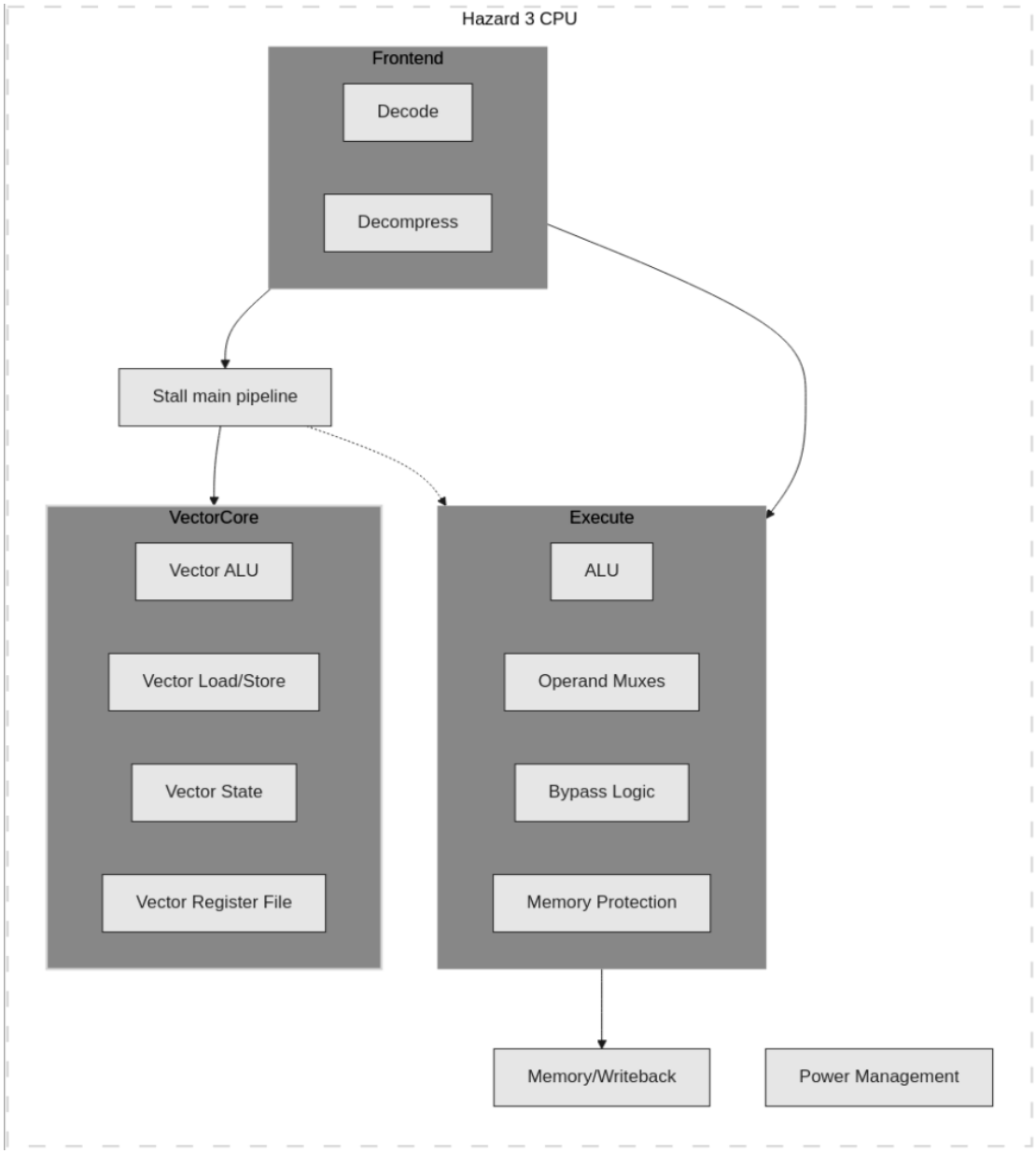


Figure 23 Final Design Flowchart

6 Ethics

6.1 Ethical Justifications

RISC-V is an open-source instruction set architecture (ISA) that empowers individuals and institutions to design their own CPUs through freely distributed and collaborative development. Unlike dominant ISAs such as x86 and ARM, which are tightly controlled through costly and restrictive licensing agreements, RISC-V promotes openness, accessibility, and innovation. Our project embraces these principles by contributing a fully open and universal Vector Extension, ensuring that anyone, whether a student, researcher, or company, can freely use, adapt, or enhance it.

To date, there are no publicly available vector extensions written in industry-standard Verilog that are open source. Our work represents a novel effort, enabling seamless integration of advanced vector processing into any RISC-V processor design. This lowers barriers to high-performance computing and fosters a culture of shared innovation.

The development of a Vector Extension significantly enhances computing across a wide range of domains. It accelerates research by enabling faster simulations and complex system modeling, thereby driving innovation in science and engineering. Economically, it improves automation, optimizes logistics, and increases productivity—contributing to sustainability and cost-effectiveness. Most importantly, by aligning with the open RISC-V ecosystem, our project helps make advanced technology affordable and accessible, empowering a wider range of developers, researchers, and small businesses worldwide.

Beyond its open-source value, vector processing is a foundational technology in emerging fields like artificial intelligence. RISC-V processors with vector extensions are already being used to train and run AI models, and their impact is expected to grow. Applications range from digital signal processing and computer graphics to AI training and inference. Yet the utility of vector processing extends far beyond these areas. It is a versatile computational paradigm with relevance to nearly every branch of modern computing.

6.2 Senior Design and Virtues of being a good engineer

RISC-V's open source ecosystem enables entrepreneurs, researchers, and companies to build upon the work of our community without being faced with licensing obstacles. Accordingly, their innovations become a contribution to the community, driving collaborative innovation.

Power consumption is a major concern in computing, with increasing demand for high performance leading to increased power consumption. Our Vector Extension aims to make the CPU more efficient by maximizing performance and minimizing the overall run time, thereby minimizing power consumption. By minimizing the time a processor has to spend on executing complex operations, we can achieve significant energy savings at the chip level. When implemented at scale in data centers, scientific computing, and AI-driven applications, such efficiency gains can translate into huge savings in global energy consumption. Not only does this innovation promote computational sustainability, but it also helps enable a sustainable future by reducing the environmental impact of high-performance computing.

One of the main goals of our project is to render our Vector Extension open to anyone who wants to use, modify, or build on it. In alignment with the foundational philosophies of RISC-V, we believe in creating an open source community that allows people to work together and make innovative advancements. Through our contribution to the RISC-V platform, we aspire to offer the next generation of students, researchers, and developers a useful foundation upon which to build their own projects and developments. Our effort is directed towards facilitating and stimulating the next wave of computing advancements, whether through academic research, industrial deployment, or further innovations.

Our team has divided up the labor of the project into week sized chunks of labor that play into our strengths. As much of our project is writing code, much of the work can be done individually. As such, we have the flexibility to work on the project during the times we’re free as long as we do our part for each week.

Our team did have some disagreements, especially in the beginning of our project when we were deciding what type of core we wanted to implement the vector extension on. We voiced our concerns and worked together to look for the right possible core and avoided conflicts of interests that would’ve wasted time.

6.3 Safety, Risk, the Public, and Informed Consent

This project is released under the Apache License, Version 2.0, with further details provided in Section 6.4. As an open-source initiative, one potential risk to the public is the possibility of privacy breaches through malicious exploitation. Due to the novel nature of our implementation, there may be undiscovered vulnerabilities that could be leveraged by attackers to modify the functionality of the vector co-processor or the Hazard3 core. Such exploitation poses a potential risk to users who download and integrate our implementation. Users are advised to exercise caution, conduct thorough security evaluations, and accept full responsibility for its use in their systems.

6.4 Legal Considerations

Our project will be released under the Apache 2.0 license, a full text of which can be found on the original apache site[3]. The license is very permissive, allowing redistribution, contribution and modification with no restrictions as long as the license is included, acknowledgement is given and patents and trademarks of the original creator are respected. We currently have not patented or trademarked any aspect of our project. A copy of the license text will be distributed with the source code and be prominently displayed.

As part of our project we used an existing project, the Hazard3 Core, which has the same license. To meet its requirements, we will acknowledge this usage and state all modifications we made, at a high level.

We also use the RISC-V vector specification, which is released under a Creative Commons BY 4.0 License and only requires attribution. We have no intention of modifying or redistributing this content, it is only for reference purposes. The RISC-V name and logo are also trademarked and require attribution. We do not plan to use them as our own, but only for informational purposes, as in this academic paper.

[3] https://www.apache.org/licenses/

7 Conclusion

Our senior design project is an incomplete implementation of the RISC-V vector extension 1.0 specification written in Verilog. Our design follows the stable 1.0 spec due to its maturity and definitive implementation definition. Our minimum viable product was to develop the working Zve32x specification for integer and fixed point operations, however we have not been able to fully implement it due to time constraints.

The project is built on the Hazard3 open source project and open sourced under the same license. We have modified the underlying core as minimally as possible, to allow for a cleaner and more maintainable implementation. A more performant implementation would likely require deeper integration, to prevent pipeline issues and greater memory bandwidth. One aspect we did change for developer experience was switching from Yosys to Verilator for simulating the core.

During the development of the load/store unit, the main challenge was figuring out the equations for finding the register and position to account for LMUL, and NF. At a glance it seems simple, until you come across edge cases, like when LMUL = 1/2, NF = 2 and loading data into operand vs1. Making sure the load store unit swapped between vs1 and vs2 correctly took quite some time. Another significant challenge was figuring out the load/store interface. Assumptions were made when initially designing the load/store state machine. We initially thought that the process of a load request was the vector core sends a request to memory and it waits for an acknowledgement. After the acknowledgement, we wait for memory to send a signal that the data we requested is now at the load store port. Turns out, when we send the memory request, we receive the “acknowledgement” in the same cycle.

During the development for the ALU, simulating and synthesizing a parallelized gate level implementation of vector integer arithmetic instructions processed in parallel for all vector elements, especially for different SEW sizes, proved to be a challenge. While a gate level implementation gave better control over the chip area and reduced any violations during synthesis, simulations produce too many errors and synthesize uses too many I/Os. Basic operands (+, -, *, /) were instead used to solve each vector one at a time as a packed array (128 bits), resolving any simulation issues and utilizing existing DSP blocks during synthesis to reduce chip area.

The current settings for the vector flags in the ALU modules work as intended. Given the time constraints, additional testing could be made for all possible flag combinations for flawless execution.

Ultimately, the biggest constraint we faced was time. We've completed implementing the instruction Decode, Load, Execute, and Store, but didn't have enough time to test and optimize our implementation to show quantitative improvements of the co-processor compared to the existing core when compared to the Dot Product and Mandelbrot Set. We also didn't have time to implement floating point and complex number instructions as our primary focus was getting our implementation of vector integer and fixed point instructions to work.

While we did face some challenges and time constraints in the development of this project and our implementation could use additional work for optimization, we offer a great starting point for future students and researchers to complete this project and to learn more about Vector Extensions.

Future students and researchers can improve our project in many ways. They can implement vector trap and error handling. They can thoroughly test the load/store unit and solve any bugs that are found, they can find a way to reduce cycles when loading 4 consecutive 8-bit elements by just loading a single 32-bit element. They can also implement pipelining to make a faster processor. They can also combine our Git branches together as the full load/store unit is in a different branch than the complete ALU units and only has basic add/sub capabilities.

Additionally, by following the Apache 2.0 license, anyone is able to use, modify, and distribute our work, adhering to the RISC-V philosophy to democratize technology.

Works Cited

[1] Marena, Ted. *The Journey of RISC-V Implementation*, 10 September 2019, https://documents.westerndigital.com/content/dam/doc-library/en_us/assets/public/western-digital/collateral/white-paper/article-journey-of-RISC-V-implementation.pdf. Accessed 7 June 2025.

[2] Novic, Kasa. "Vec Ext 1.0." *Vector Extension 1.0, frozen for public review*, 19 September 2021, https://github.com/riscvarchive/riscv-v-spec/releases/tag/v1.0. Accessed 7 June 2025.

[3]. "DSP48E2." *DSP48E2 - 2020.2 English - UG958*, https://docs.amd.com/r/en-US/ug958-vivado-sysgen-ref/DSP48E2. Accessed 7 June 2025.

[4]. "Qualcomm." *Qualcomm to Bring RISC-V Based Wearable Platform to Wear OS by Google*, 17 October 2023, https://www.qualcomm.com/news/releases/2023/10/qualcomm-to-bring-risc-v-based-wearable-platform-to--wear-os-by-. Accessed 7 June 2025.

[5]. "RISC-V." *How NVIDIA Shipped One Billion RISC-V Cores In 2024*, 25 February 2025, https://riscv.org/blog/2025/02/how-nvidia-shipped-one-billion-risc-v-cores-in-2024/. Accessed 7 June 2025.

[6]. "RISC-V2." *RISC-V 2 : A Scalable RISC-V Vector Processor*, ResearchGate, October 2020,

https://www.researchgate.net/publication/349293112_RISC-V_2_A_Scalable_RISC-V_Vector_Processor. Accessed 7 June 2025.

[7]. “What is Electronic Design Automation (EDA)? – How it Works.” *Synopsys*, https://www.synopsys.com/glossary/what-is-electronic-design-automation.html. Accessed 7 June 2025.

[8] Mujtaba, Khondakar. “Hazard3Vec.” *Hazard3Vec*, January 2025, https://github.com/kmarzouq/Hazard3Vec. Accessed June 8 2025.

[9] Intel, “Intel® Instruction Set Extensions Technology,” Intel Corporation. Accessed: Mar. 30, 2025. [Online]. Available: https://www.intel.com/content/www/us/en/support/articles/000005779/processors.html

[10] 754_WG - Working Group for Floating-Point Arithmetic, “IEEE Standards Association,” IEEE Standards Association. Accessed: Mar. 30, 2025. [Online]. Available: https://standards.ieee.org/ieee/754/6210/

[11] RISC-V International, “GitHub - riscvarchive/riscv-v-spec: Working draft of the proposed RISC-V V vector extension,” GitHub. Accessed: Mar. 31, 2025. [Online]. Available: https://github.com/riscvarchive/riscv-v-spec

[12] RISC-V International, “About RISC-V International,” RISC-V International. Accessed: Mar. 31, 2025. [Online]. Available: https://riscv.org/about/

[13] K. Patsidis, C. Nicopoulos, G. Ch. Sirakoulis, and G. Dimitrakopoulos, “RISC-V2: A Scalable RISC-V Vector Processor,” in 2020 IEEE International Symposium on Circuits and Systems (ISCAS), IEEE, Oct. 2020, pp. 1–5. Accessed: Mar. 31, 2025. [Online]. Available: https://doi.org/10.1109/iscas45731.2020.9181071

[14] M. Perotti, M. Cavalcante, N. Wistoff, R. Andri, L. Cavigelli, and L. Benini, “A ‘New Ara’ for Vector Computing: An Open Source Highly Efficient RISC-V V 1.0 Vector Processor Design,” in 2022 IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP), IEEE, Jul. 2022, pp. 43–51. Accessed: Mar. 31, 2025. [Online]. Available: https://doi.org/10.1109/asap54787.2022.00017

[15] J. Zhao et al., “The Saturn Microarchitecture Manual,” EECS Department, University of California, Berkeley, 0 2024. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2024/EECS-2024-215.html

[16] Chips Alliance, “Torrent 1,” Mar. 2025. Accessed: Mar. 31, 2025. [Online]. Available: https://github.com/chipsalliance/t1

[17] Intel Labs, “GitHub - IntelLabs/riscv-vector: Vector Acceleration IP core for RISC-V*,” GitHub. Accessed: Mar. 31, 2025. [Online]. Available: https://github.com/IntelLabs/riscv-vector

[18] SiFive Inc, “P270 Data Sheet,” 2022. Accessed: Mar. 31, 2025. [Online]. Available: https://www.sifive.com/document-file/p270-and-p270-mc-data-sheet

[19] The SHD Group, “RISC-V Market Report: Application Forecasts in a Heterogeneous World,” The SHD group, Jan. 2024.